

Diseño Detallado de Software

Profesora María Fernanda Sepúlveda

Por Nicolás Balbontín

¿Qué es el diseño detallado de software?

El **ciclo de vida de un proyecto de Software** varía dependiendo de la metodología (como Ágil, Cascada, Espiral, entre otras). Pero todas mantienen en común algunas actividades como: comunicación, planificación, **modelación**, construcción e implementación.

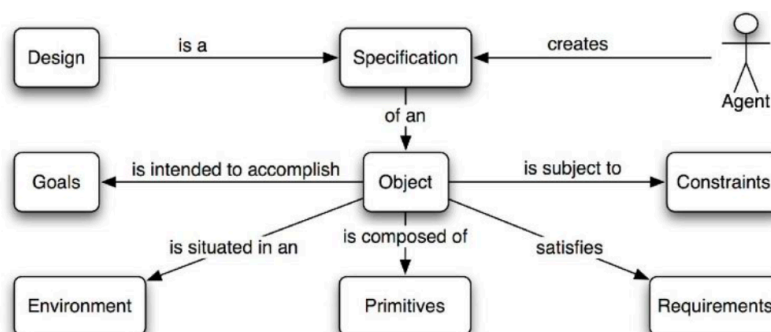
La **modelación** implica:

Análisis del problema → entender el problema no necesariamente desde una vereda “programática”. Es importante comprender el **contexto** y el **negocio**.

Diseño de la solución → determinar y guiar a cómo resolver el problema. Se deciden los **componentes**, la **forma de los datos** y la **arquitectura del sistema**.

* Este curso consiste en generar las herramientas de forma integral para poder diseñar de forma detallada software que resuelva un problema y que perdure en el tiempo. *

Importancia del Diseño



Es **más barato corregir** un error al momento de **idear** su concepción que al momento de **programar**. La fase de diseño es el momento en que la calidad del código se establece.

❓ ¿Qué pasa si se omite el diseño en todo el proceso? **Corregir los errores toma tiempo.** Los problemas típicos que comienzan a aparecer son:

- Hacks. Deuda técnica.
- Omisión de casos bordes: bugs.
- Omisión de requisitos esenciales si se trabaja con clientes o productos externos. Errores típicos en cascada o soluciones no iterativas.
- Retrasos. Si no se tiene un *road map* de lo que se va a hacer se pueden tener estimaciones muy optimistas.
- Dificultad en integrar, mantener y extender.
- Dependencia de miembros específicos del equipo. El programador del producto es el único que entiende las mañas, por lo que la documentación pasa a ser indispensable.
- Imposibilidad de testing.
- Alejamiento de la necesidad del cliente.

Diseño en el proceso de desarrollo

En **metodologías ágiles**, se espera que la fase de diseño se mezcle con el proceso de desarrollo en distintas etapas.

- Se **vuelve al iterar sobre el diseño** en la medida que se aterriza la conceptualización de la solución que se construye y el código que se escribe.
- Idealmente, diseño es una tarea que queremos hacer en equipo.
- Desarrollos que tengan sus etapas muy definidas, diferencian la etapa de análisis y de diseño, donde el análisis es una versión simplificada o menos estructurada del diseño. La primera etapa es entender el problema y los requisitos del usuario y luego poner las manos a la obra.
- Una herramienta usada para el diseño son **diagramas**.

Podemos dividir el diseño en 4 área:

- **Diseño de Arquitectura:** Abarca la representación de las estructuras de datos y los componentes de software necesarios para la construcción. Implica también la interacción entre las distintas tecnologías del proyecto.

- **Diseño de Componentes:** La arquitectura de un sistema define cuáles tipos de componentes mínimos debe tener. Durante el diseño de componentes se busca concretizar los componentes seleccionados, aportando mayor precisión sobre sus estructuras de datos, algoritmos, interfaces y mecanismos de comunicación; dividiéndolos en sub-componentes con responsabilidades más acotadas. No definen qué funciones tienen pero sí qué **componentes** hay y cómo se **comunican**.
- **Diseño de Interfaces:** Comunicación con sistemas externos y humanos.
 - Diseño de Interfaces Gráficas (UI)
 - Diseño de API's externas
 - Diseño de API's internas

Algunos diagramas utilizados:

 - *Wireframes*: Diseño en baja fidelidad de la vista de la aplicación que se está desarrollando. No tiene porqué ser una especificación del diseño final.
 - Diagrama de flujo: Explica cómo son las páginas, opciones de salida, qué es lo que debe aparecer en cada una de las vistas y cómo uno se mueve entre las diferentes vistas de la aplicación.
 - Otros: Flowchart, Communication diagram, etc
- **Diseño de Clases/Datos:** Expone granularmente lo que es la aplicación. Esquema de clases y sus relaciones. Ejemplo: Diagrama de clases, diagrama ER. Son más escasos debido a que aplicaciones muy grandes es demasiado el detalle que sería necesario implementar.

Principios fundamentales

El objetivo de la fase de diseño es **garantizar la calidad** de la solución

- **Abstracción:** rescatar solamente la **información relevante** dado el **contexto**. Debe cumplir con criterios de **suficiencia** y **completitud**. No es necesario agregar tanto detalle a algo que con poca información se entiende - principio de completitud: debe ser completa para que se entienda lo que se está haciendo pero no excesivamente detallado-.

- **Encapsulamiento o Ocultamiento:** no exponer información o lógica innecesaria. Por ejemplo, servicios web - cookie de *login*- , librerías y módulos con modificadores de acceso - exposición de la base de datos -. . Con esto se logra **mantenibilidad, reusabilidad y extensibilidad**.
- **Cohesión:** una **clase** tiene un conjunto **preciso y acotado de responsabilidades** y todos sus atributos y métodos están únicamente relacionados con cumplir esas responsabilidades. Beneficios de reducir **complejidad** y aumentar **mantenibilidad**. No es necesario asignarle responsabilidad a un módulo o componente más grande que no deba tener. Ejemplo: el módulo de usuario no debería tener la lógica de contabilidad.
- **Acoplamiento:** el acoplamiento es una medida de cuán conectados están dos subsistemas o clases. Si existe una **alto acoplamiento** un módulo modifica o se apoya en el funcionamiento interno de otro módulo. Qué tanto un módulo depende del otro. Si existe demasiada dependencia, un cambio en un módulo puede generar un colapso en el otro. Genera dificultad en la testeabilidad.

Design Quality Guidelines

1. Exhibe una arquitectura con **patrones reconocidos**. Es innecesario reinventar patrones que ya existen.
2. Es **modular**, particionado lógicamente en subsistemas. Tiene que ser cohesionado y con bajo acoplamiento.
3. Contempla datos, arquitectura, interfaces y componentes.
4. Permite derivar estructuras de datos óptimas para el problema en cuestión.
5. Revela componentes con características funcionales **independientes**. Bajo acoplamiento.
6. Revela **interfaces sencillas** que minimizan la complejidad de interactuar con el sistema.
7. Se deriva lógicamente del análisis de los requisitos del proyecto. Un diseño bueno debe responder a las necesidades del proyecto sin la necesidad de que este requiera conocimientos técnicos. Hace sentido utilizar los componentes.
8. Utiliza una notación estándar para comunicar su significado.

¿Qué es UML?

"Unified Modeling Language (UML) es la familia de notaciones gráficas, respaldadas por un modelo, que ayudan a describir y diseñar sistemas de software, enfocado principalmente a sistemas orientados a objetos" (Martin Fowler, 2003. Principales figuras en el ámbito de desarrollo de software).

El valor principal de UML es la comunicación y el entendimiento. Está pensado tanto para especialistas como para no-expertos, cualquiera puede entenderlo. A pesar de que las reglas estándar (por la OMG) son rigurosas, los diagramas suelen ser modificados dependiendo de las necesidades y experiencia. UML es solo diagramación. No existe un *mapping* oficial directo con el código, es decir, no está pensado para ser programado, sino que para ser **comunicado**.

¿Cuándo usamos UML?

UML está pensado para **objetos**. Pueden ser usados como:

- **forward engineering**: realizar una planificación exhaustiva antes de programar.
- **reverse engineering**: realizar UML una vez que ya existe el sistema, a modo de explicar lo que hay o para reorganizar lo que se creó.

Utilización:

- UML como un **blueprint**. Es más formal. Pensado para definir los requerimientos o documentación formal del sistema. Generalmente por **expertos**. Es difícil realizar un *forward engineering* si no se tiene experiencia. Asociado a **forward engineering**.
- UML como **sketch**. Esto es usado entre programadores para comunicar y expresar ideas. En el proyecto será utilizado de esta forma. Muy usado.
- UML como **lenguaje de programación**. Poco común porque se requieren herramientas especializadas. No existe un estándar para el paso diagrama a código.

¿Cómo clasificamos los diagramas UML?

- Diagramas de **Comportamiento**. Qué se comunica con qué, interacción y comunicación más que la estructura.
- Diagramas de **Estructura**. Organización de solución. Foto de la estructura en un momento dado.

Modelo 4+1 propone otra forma de clasificar los diagramas.

Nombre	Propósito	Nombre	Propósito
1. De Actividad (Activity)	Comportamiento paralelo y de procedimientos.	8. De Objetos (Object)	Ejemplo de configuraciones e instancias.
2. De Clases (Class)	Clases, funcionalidades y relaciones.	9. De Paquetes (Package)	Estructura jerárquica en tiempo de compilación.
3. De Comunicación (Communication)	Interacción entre los objetos. Énfasis en los <i>links</i> .	10. Secuencia (Sequence)	Interacción entre objetos. Énfasis en la secuencia.
4. De Componentes (Component)	Estructura y conexión entre componentes.	11. Máquina de estados (State machine)	Como los eventos cambian un objeto a lo largo del ciclo.
5. Estructura Compuesta (Composite structure)	Descomposición de una clase en tiempo de ejecución.	12. De Tiempos (Timing)	Interacción entre objetos. Énfasis en el tiempo.
6. De Despliegue (Deployment)	Despliegue de los <i>artefactos</i> hacia los <i>nodos</i> .	13. De Casos de Uso (Use case)	Cómo los usuarios interactúan con el sistema.
7. Global de Interacciones (Interaction overview)	Mezcla de secuencia y actividad.	14. De Perfil (Profile)	Define estereotipos, clasificaciones y restricciones.

1. De Actividad (Activity)

Se conoce como diagrama de flujo. Modela procesos en base a reglas. Similar a VPNM

2. De Clases (Class)

Los diagramas de clase buscan **describir** los distintos **objetos** presentes en un sistema, cómo **interactúan** y el conjunto de **atributos** y **métodos** de cada uno de ellos.

3. De Comunicación (Communication)

Modela la **comunicación** y **dirección** de los mensajes entre **clases**. Pone particular énfasis en los flujos. La gracia es que habla más de **funciones**. Cada línea es una función. Los componentes no necesariamente hacen 1 a 1 con el sistema. Enumera el

funcionamiento. El diagrama de actividad es más común que este. Cómo se estructura el código.

4. **De Componentes** (*Component*)

Modela la **dependencia entre los componentes** del software.

Los componentes son **aspectos del diseño** de una solución de software más grandes y abstractos que una clase. También se dice que modela la **dependencia** entre los componentes. La gracia es que expone quién expone qué y quién se alimenta de.

Más abstracción que el diagrama de clases.

5. **Estructura Compuesta** (*Composite Structure*)

Modela la **estructura interna** o colaboración **entre** los **elementos** de una **clase**. Generalmente se usa para complementar otros diagramas. Es como hacer un doble *click* a un **diagrama de clases**. Las partes pueden ser una clase completa o abstracta. Poco usado.

6. **De Despliegue** (*Deployment*)

También conocido como diagrama de **arquitectura**. Ilustra la ubicación física de cada componente de una solución de software compleja. Se definen:

- **Nodos:** quién ejecuta el software, puede ser un hardware, una máquina, una máquina virtual, un contenedor, etc.-. Se pueden definir nodos dentro de nodos.
- **Artefactos:** el *build*, por ejemplo, la aplicación.
- **Canales de comunicación:** vía o protocolos por los cuales se comunican los distintos nodos y artefactos.

Está pensado para los que tienen que levantar la aplicación

7. **Global de Interacciones** (*Interaction Overview*)

Similar al diagrama de actividad, con la diferencia que las actividades pueden ser representadas como **diagramas de actividad** (nesteado). La notación es la misma que la del diagrama de actividades. Esto es como hacerle un doble *click* a un **diagrama de actividad**.

8. De Objetos (*Object*)

Diagrama que muestra un **snapshot** de una parte de la aplicación, ilustrando objetos instanciados presentes y sus referencias hacia otros objetos. Los atributos de estos objetos pueden estar total o parcialmente definidos. Este es un diagrama de **instancia** de las **clases**. Sirve para ver cómo se diferencian las instancias. Definir todas las instancias posibles de una clase y cómo se comportarían.

9. De Paquetes (*Package*)

A medida que un **sistema crece**, puede volverse caótico gestionar el gran número de clases que define y se vuelve necesario **agruparlas** de manera lógica. Los paquetes en UML se usan principalmente para agrupar **conjuntos de clases**, pero pueden usarse con otros componentes de UML también.

Común cuando se utiliza Java ya que este tiene muchas librerías *webiadas*. Se concatenan todos los nombres del paquete. *Fully qualified class name* es el más usado.

10. Secuencia (*Sequence*)

Permite **describir gráficamente** transacciones (conjunto de operaciones) complejas entre múltiples agentes y cómo interactúan entre ellos. Este diagrama se ve harto de manera comunicacional. Debe ser el diagrama más común. El tiempo siempre va hacia abajo. La línea es la dirección de la secuencia. Trata de describir cómo se comporta la aplicación o sistema en cuánto al tiempo. Las líneas punteadas solo son opciones mientras que las cajas habla de una rutina en sí.

11. Máquina de Estados (*State Machine*)

O Diagrama de Estados. Permiten definir el **comportamiento de un sistema**. Definen dos componentes esenciales: **Estados** y **Transiciones**. Describen como un sistema puede ir **cambiando de estado** a medida que se **cumplan** las **condiciones** delineadas en las transiciones. La orientación es similar al diagrama de actividad solo que con menos elementos.

12. De Tiempos (*Timing*)

Permiten poner el énfasis en los **cambios de estado** de un sistema a medida que avanza el tiempo. Son muy útiles para **representar interacciones complejas** gatilladas solo por el paso del tiempo. Aunque también soportan comunicación entre participantes mediante estímulos y paso de mensajes, no son aptos para ilustrar de manera legible comunicación en gran volumen. Poco común.

13. De Casos de Uso (*Use Case*)

Estándar para graficar conjuntos de casos de uso de forma breve. Ilustra los **nombres** de los casos de uso, **actores** y **relaciones** entre ellos. Agrupan múltiples escenarios posibles de **forma sintetizada**. Importante en la ingeniería de software.

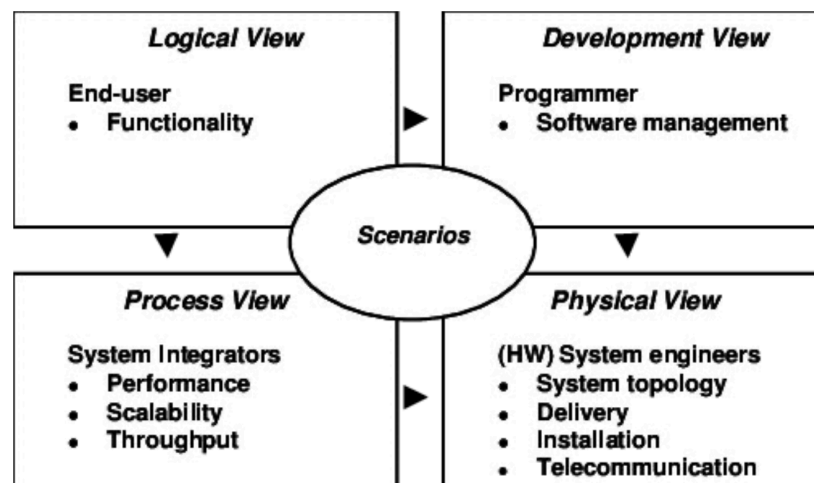
14. De Perfil (*Profile*)

Mecanismo para extender los modelos propuestos por UML a dominios o tecnologías específicas con elementos customizados. Enfocado a lo que se habla de lenguaje de programación en UML. Diagrama de clase combinado con paquetes.

Modelo 4+1

Framework para describir la arquitectura de un software, basado en el uso de múltiples vistas concurrentes. Propuesto por Philippe Kruchten en 1995. Se basa en **vistas**. Una vista es una **representación del sistema completo**, enfocada desde una perspectiva con consideraciones específicas.

UML divide los diagramas de **estructural y comunicación**, el modelo 4+1 trata de dar otra perspectiva de cómo dividirlos y cómo presentarlos. Estas son vista lógica, vista de procesos, vista de implementación, vista física y vista de escenarios.



> Vista lógica (*Logical View*)

Interesados: Usuarios finales.

Consideraciones: Requisitos Funcionales.

Objetivo: ¿Qué es lo que el sistema debería **proveer** a sus usuarios?

Diagramas asociados: Diagrama de Clases, Diagrama de Comunicación, Diagrama de Secuencia, Diagrama de Estados.

> Vista de Procesos (*Process View*)

Interesados: Analistas.

Consideraciones: Requisitos No Funcionales.

Objetivo: ¿Qué procesos y reglas tiene el sistema, y cómo interactúan los componentes?.

Diagramas asociados: Diagrama de Actividad.

> Vista de implementación

Interesados: Desarrolladores y Líderes de proyectos.

Consideraciones: Organización del Software.

Objetivo: ¿Cómo se organiza el software del sistema?

Diagramas asociados: Diagrama de Componentes, Diagrama de Paquetes (arquitectura).

> Vista física

Interesados: Arquitectos de Sistemas.

Consideraciones: Requisitos No Funcionales.

Objetivo: ¿Cómo es el ambiente de ejecución del sistema?

Diagramas asociados: Diagrama de Despliegue.

> Vista de escenarios

Interesados: Todos.

Consideraciones: Consistencia y validez.

Objetivo: ¿Qué es lo que el sistema debería hacer?

Diagramas asociados: Diagrama de Casos de Uso.

Para documentar sus programas, las empresas lo usan como un estándar pensando en estas 4 vistas.

"Small things matters"

Es un dicho ampliamente popular. Sin embargo no le quita lo veraz.

La práctica en los **detalles** permite al **profesional** mayor dominio y confianza en su trabajo. Por otra parte, un fallo en construcción genera un desencanto por el trabajo en general (Robert C. Martin, 2008).

Total Productive Maintenance (TPM)

Eliminación de pérdidas asociadas con la detención, calidad y costes en los procesos de producción industrial. Habla de 5 conceptos:

- **Organización:** Convenciones de nombre.
- **Limpieza:** Mantener el espacio de trabajo libre de desechos. Si una línea de código no aporta, **elimínala**. Sacar las cosas que son inútiles. Un *pull-request* que elimina más líneas de las que agrega es bueno.
- **Ordenar, sistematizar:** Una pieza de código debe estar donde esperas encontrarlo. Si no es así, **refactor**.
- **Disciplina:** Tener disciplina para seguir las prácticas, reflejar en el trabajo y voluntad de **adaptarse**.
- **Estandarización:** Como grupo se llega a un **consenso** acerca de cómo mantener el espacio limpio.

TPM apunta a que no siempre hay que apuntar a **producir** a velocidad óptima sino que a crear máquinas que se **mantengan mejor**.

Las buenas prácticas son todas las que nos permitan crear **código mantenible**.

- El código mantenible es igual de importante que el código ejecutable.
- Se debe apuntar a criterios de robustez, extensibilidad y mantenibilidad.
- No es lo mismo escribir **código limpio** para un proyecto sencillo, que para un proyecto complejo compuesto de varios componentes obligados a cooperar

Principios SOLID

Propuestos por Robert C. Martin (cerca del 2000), en el libro "Agile Principles, Patterns and Practices in C#". Constituyen un set de estándares de alto nivel para construir código de calidad en lenguajes orientados a objetos.

Estos principios son:

- **Single-responsability principle** (SRP): Una clase debe tener solo una responsabilidad. Relacionado con el concepto de **cohesión** (alta cohesión y bajo acoplamiento).
- **Open-closed principle** (OCP): Establece que una clase debiese estar **abierta** a **extensión** y **cerrada** a **modificación**. No debiera modificarse el código interno de una clase una vez que está creada y siendo utilizada. No es necesario abstraer todo de inmediato, prefiere refactorizar cuando sea necesario. Utilizar clases intermedias para mediar las conexiones.
- **Liskov substitution principle** (LSP): Establece que todo **subtipo** debe ser posible **sustituirlo** por un **supertipo** sin introducir comportamiento errático o anti-intuitivo en el programa. Supertipo es cuando se tiene herencia.
- **Interface segregation principle** (ISP): **Desventaja de interfaces con demasiados métodos**. Clases que implementan interfaces no deberían estar forzadas a requerir implementaciones de métodos irrelevantes para ellas. Muchas veces resulta deseable descomponer las interfaces grandes en varias interfaces más pequeñas.
- **Dependency Inversion principle** (DIP): Establece que **módulos de alto nivel** no deben **depender** de módulos de **bajo nivel**; ambos deben **depender** de **abstracciones**. Principio orientado a **facilitar** el proceso de **testing** y extensión en las clases que definamos. Una forma para cumplir con el principio es usando el patrón de diseño **Inyección de Dependencia** (*Dependency Injection*): En vez de crear el objeto en la clase, se le suministra el objeto a través del constructor, una propiedad o un método (de ahí Injection).

Calidad

Garvin (1984) define la calidad según vistas:

Vista trascendental: la calidad se percibe, pero no se puede explicar.

Vista del usuario: la calidad en base a los objetivos del usuario final.

Vista del productor: la calidad según las especificaciones del producto.

Vista del producto: la calidad en función de lo que hace el producto.

Vista del valor: la calidad en base a lo que está dispuesto a pagar un consumidor.

Pressman (2009) lo define como: "Un desarrollo de software efectivo, aplicado de una manera que crea un producto útil que provee valor cuantificable para aquellos que lo producen y aquellos que lo utilizan"

Métricas de calidad: valores objetivos, sencillos y fáciles de computar, que además cumplen con ser consistentes, con unidades de medición expresivas, independientes del lenguaje de programación y reflejan recomendaciones para mejorar la calidad del software. No existe una métrica correcta.

- **Métrica Complejidad ciclomática (Cyclomatic Complexity):** Es una métrica basada en el cálculo del número de **camino independientes** que tiene el código. Apunta a la testeabilidad y complejidad.
- **ABC Size:** También mide complejidad.
- **Depth of Inheritance Tree (DIT):** Es la máxima profundidad de una clase en su árbol de jerarquía de clases. Una clase que no tiene ninguna superclase tiene DIT= 1, los hijos de una clase tienen DIT=2, los hijos de los hijos de una clase tienen DIT=3. A medida que aumenta el DIT se vuelve más complicado entender el comportamiento de una clase dado que requiere entender y visualizar más y más capas de herencia. Hay un trade-off con la reutilización del código: mayor repetición pero mayor complejidad.
- **Number of Children (NOC):** Cantidad de subclases directas de una clase. Refleja el nivel de abstracción del diseño. Indica el riesgo.

- **Coupling Between Objects (CBO):** Cuántas asociaciones se cuentan entre clases. Minimizar asociaciones.
- **Response for Class (RFC):** Cantidad de métodos únicos invocados de una clase. Es más normal que una clase tenga métodos privados que públicos, ya que si tiene muchos públicos tiene muchas responsabilidades, complejizándolo.
- **Lack of Cohesion Of Methods (LCOM):** Cantidad de grupos de métodos que acceden a 1 o más atributos en común de una clase.

Code Smells

Indicador común superficial que por lo general corresponde con un problema más profundo en el sistema. **No son bugs:** el programa funciona correctamente, pero su débil diseño dificulta el desarrollo e incrementa la posibilidad de generar bugs.

5 tipos:

- **Bloaters:** Representan código, métodos y clases que han crecido a tal punto que es difícil trabajar con ellos.
- **Long Method:** Líneas excesivas de código. Si tiene más de 10 puede haber un problema.
- **Large Class:** Una clase contiene excesivas líneas de código, atributos y métodos. Se puede descomponer usando asociación, herencia.
- **Primitive Obsession:** Uso de variables primitivas en vez de objetos pequeños. Uso de constantes para guardar información.
- **Long Parameter List:** Más de 3 o 4 parámetros por método. Es difícil entender listas extensas de parámetros.
- **Data Clumps:** Diferentes partes de una aplicación contienen una definición recurrente de variables.
- **Object-Orientation Abusers**
- **Change Preventers**
- **Dispensables**
- **Couplers**

Platanus

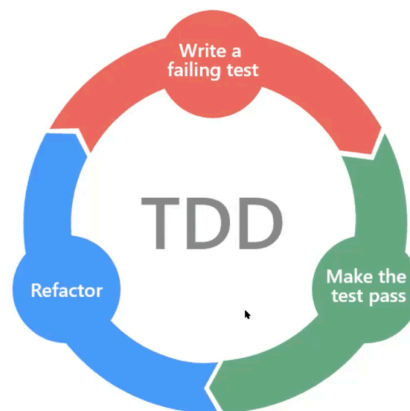
Incertidumbre

Ganar en programación es resolver un problema.

¿Qué problema estamos resolviendo? ¿Quiénes son las personas que tienen ese problema? ¿Cómo atacar el objetivo? ¿Cómo atacar los algoritmos complejos?

Ejemplo: Simulador de tasas de crédito hipotecario.

- Partir por lo **más incierto**:
 - Integración con una API.
 - Un lenguaje nuevo.
 - Migración de datos compleja.
- **UX**



Test Driven Development: Manera iterativa para desarrollar una función de algo. Programar primero el test que falla para luego programar el código para que pase el test y finalmente hacerle un refactor.

Ejemplo: Algoritmo que resuelve el vuelto óptimo en monedas de \$1, 5, 10, 50 y 100.

Partir por lo más fácil/certero

1. Resolver el vuelto de \$1.
2. Resolver el vuelto de \$3.
3. Resolver el vuelto de \$5.
4. Resolver el vuelto de \$8.

5. Resolver el vuelto de \$10.
6. Resolver el vuelto de \$18.

7. Resolver el vuelto de \$157.

Privilegiar un código **entendible** a un código **super optimizado**.

Clase 3: Buenas Prácticas de Desarrollo

7 de Octubre, 2020

Code Smells

- **Object-Orientation Abusers:** Implementación incompleta o incorrecta de los principios de programación orientados a objetos.
- **Switch Statements:** Se tiene un operador switch complejo o una secuencia de if-else. Esto es un mal uso de polimorfismo y representa una poca organización del código.
- **Temporary Field:** Se tienen atributos de una clase que solo tienen valor bajo ciertas circunstancias, estando el resto del tiempo sin definir o nulos. Se suelen usar atributos de clase en vez de pasar los datos como parámetros.
- **Refused Bequest:** Una subclase utiliza algunas propiedades y métodos de sus padres. Se quiso reutilizar código entre una superclase y una clase, pero son completamente distintas.
- **Alternative Classes with Different Interfaces:** Dos clases realizan funcionalidades idénticas, pero con nombres de métodos distintos. Esto suele suceder por un desconocimiento por parte del programador sobre la existencia de la otra clase.
- **Change Preventers:** Reflejan un posible problema si un cambio en una parte del código implica varios cambios en otras secciones, aumentando el costo de desarrollar.
- **Divergent Change:** Cambios en la clase desencadenan cambios en métodos que no tienen relación entre sí. Esto suele suceder por una estructura poco pensada.
- **Shotgun Surgery:** Cambios en la clase desencadenan cambios en otras clases. Suele suceder por responsabilidades que fueron divididas entre varias clases.

- **Parallel Inheritance Hierarchies:** Hay un exceso de dependencia en el comportamiento: Añadir una subclase A a una clase B, implica añadir una subclase C a una clase D. Cuando las jerarquías son pequeñas, las modificaciones necesarias se tienen bajo control. Pero a medida que el árbol de jerarquías crece, se hace cada vez más difícil realizar cambios.
- **Dispensables:** Fragmentos de código que son innecesarios e inútiles, además su ausencia haría que el código fuese más claro, eficiente y fácil de entender.
 - **Comments:** Un método con excesivos comentarios.
 - **Código duplicado:** Fragmentos de código idénticos.
 - **Lazy Class:** Una clase no cumple un rol que realmente justifique su existencia.
 - **Data Class:** Una clase contiene solamente atributos, sin aplicar comportamiento.
 - **Dead Code:** Una variable, parámetro, campo, método o clase ya no se utiliza.
 - **Speculative Generality:** Una variable, parámetro, campo método o clase que no se utiliza, por se planea usar.
- **Couplers:** Exceso de acoplamiento en el código.
 - **Feature Envy:** Un método utiliza más información de un objeto que recibe, que del mismo objeto en el que está definido.
 - **Inappropriate Intimacy:** Una clase utiliza métodos y atributos internos de otra clase.
 - **Message Chains:** Excesivas llamadas a métodos en cadena: a->b()->c()->d().
 - **Middle Man:** La única funcionalidad de la clase es delegar trabajo sin agregar funcionalidad.
 - **Incomplete library Class:** Una librería presenta problemas y no responde a las necesidades del problema.

Refactoring

Es el proceso de cambiar un sistema de software de tal forma que no se altera el comportamiento externo, pero que **mejora la estructura interna del código**.

La forma más eficiente de hacer refactoring es teniendo buenos tests acompañando al código.

¿Por qué hacer refactor?

Mejora el diseño de software, hace el código más fácil de entender, ayuda a encontrar bugs, ayuda a ganar experiencia y al equipo a programar más rápido.

¿Cuándo hacer refactor?

Rule of three: *three strikes and you refactor.*

La primera vez que haces algo, solo térmalo.

La segunda vez que haces algo similar, te preocupas por la duplicación pero puedes dejar pasar.

La tercera vez, haz refactor.

Cuando añadas algo nuevo, considera el ROI.

Cuando corriges *bugs*.

Cuando tu código vaya a pasar por *code review*.

Metáfora de los "Dos Sombreros"

Un desarrollado debería dividir su tiempo en 2 actividades: añadir funcionalidades y hacer refactoring:

- Dentro de las funcionalidades, solo se añaden nuevas funciones y test para medir el avance.
- Cuando se hace refactor, no se añaden más tests, solo se re-estructura el código.

Durante el desarrollo, el desarrollador estará cambiando de sombreros de forma constante. Sin embargo, es importante para el mismo percatarse cuál sombrero tiene puesto.

Problemas

- Managers poco técnicos no le dan valor.
- No hacer bien el refactoring puede causar cambios en el comportamiento.
- Hacer refactoring sobre código "no estable" trae más problemas que beneficios.
- Hacer refactoring encima de un *deadline* puede traer complicaciones.

Testing

¿Por qué falla el software?

- Porque los requisitos están mal
- Diseño del software defectuoso en un inicio.

¿Qué es testing? Son pruebas que permiten **validar** y **verificar** el funcionamiento del software.

El testing es la herramienta que permite hacer refactoring que el cambio que se haga en el código no falle.

Clase 4: Patrones de diseño y arquitectura

16 de Octubre, 2020

Cada patrón define un problema que ocurre una y otra vez en un ambiente. Es una forma de que la solución pueda ser usada tantas veces.

Un patrón de diseño se caracteriza por **3 componentes**:

- **Contexto**
- **Problema**
- **Solución**

Según propósito

- **Creacionales**
- **Estructurales**
- **De comportamiento**

Según Scope:

- **Clase**
- **Objeto**

		Propósito		
		Creacional	Estructural	De comportamiento
Scope	Clase	- Factory Method	- Adapter	- Interpreter - Template Method
	Objeto	- Abstract Factory - Builder - Prototype - Singleton	- Adapter - Bridge - Composite - Decorator - Facade - Flyweight - Proxy	- Chain of Responsibility - Command - Iterator - Mediator - Memento - Observer - State - Strategy - Visitor

www.dofactory.com

- **Factory method:** Delega una interfaz para la creación de un objeto, pero delega a las subclases que deciden qué clase instanciar. Una fábrica de métodos, es decir, la clase original te define cómo crear las cosas. Aprovecha de reutilizar código.
- **Abstract Factory:** Provee una interfaz para la creación de familias de objetos relacionadas o dependientes sin especificar sus clases concretas. Utiliza mucho las clases abstractas.
- **Builder:** Separa la construcción de un objeto complejo de su representación, para que así el mismo proceso de construcción pueda crear diferentes representaciones.
- **Prototype:** Especifica los tipos de objetos a crear utilizando prototipos, y crea objetos nuevos a través de copias de estos prototipos.
- **Singleton:** Garantiza que una base tenga solamente una instancia y provee un acceso global a la instancia. Es común y no tan querido ya que a veces es antipatrón, conduciendo a más problemas que soluciones.
- **Adapter:** Convierte la interfaz de una clase en una interfaz que sea esperada por el cliente. Adapter permite a clases distintas interactuar, que sin su interfaz común serían incompatibles. Típico cuando se utilizan sistemas externos o muchos clientes distintos.
- **Bridge:** Desacopla una abstracción de su implementación, para que ambas puedan variar independientemente.
- **Composite:** Compone objetos en estructuras de árboles para representar jerarquías completas. Composite permite a los clientes tratar objetos y composiciones de objetos de igual manera. Sirve cuando se quieren armar **jerarquías**.
- **Decorator:** Agrega responsabilidad adicional a un objeto de manera **dinámica**. Decoradores permiten una alternativa flexible para extender funcionalidad de subclases.
- **Facade:** Provee una interfaz unificada a un conjunto de interfaces en un sub-sistema. Fachada define una interfaz de alto nivel que hace a un sistema más fácil de usar. Se tiene una interfaz intermedia para responder a las clases. Cuando Facade crece demasiado puede convertirse en un antipatrón.
- **Flyweight:** Utiliza valores por referencia para soportar grandes números de objetos.

- **Proxy:** Provee un sustituto para que ese objeto sustituto controle el acceso al original.

Clase 5: Arquitectura

6 de Octubre, 2020

En el mundo del desarrollo, se separan las nociones de arquitectura de software del diseño de software, en que la arquitectura busca una **abstracción general** del sistema que se construirá, y define las prioridades de la solución respecto a los atributos de calidad esenciales.

Repaso

		Propósito		
		Creacional	Estructural	De comportamiento
Scope	Clase	- Factory Method	- Adapter	- Interpreter - Template Method
	Objeto	- Abstract Factory - Builder - Prototype - Singleton	- Adapter - Bridge - Composite - Decorator - Facade - Flyweight - Proxy	- Chain of Responsibility - Command - Iterator - Mediator - Memento - Observer - State - Strategy - Visitor

Patrones creacionales: Los patrones creacionales proporcionan varios mecanismos de creación de objetos que incrementan la flexibilidad y la reutilización del código existente.

Factory Method: Generar una clase abstracta que posea un método de tal forma que las clases que hereden de esa clase puedan utilizar ese método.

Abstract Factory: Proveer una interfaz que posea una lista de métodos para la creación de familias de objetos relacionados o dependientes sin especificar sus clases concretas.

Patrones Estructurales: Los patrones estructurales explican cómo ensamblar objetos y clases en estructuras más grandes, a la vez que se mantiene la flexibilidad y eficiencia de estas estructuras.

Adapter: permite la colaboración entre objetos con interfaces incompatibles.

Composite: permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.

Facade: proporciona una interfaz simplificada a una biblioteca, un framework o cualquier otro grupo complejo de clases. Resulta útil cuando se tiene que integrar la aplicación con una biblioteca con decenas de funciones, y solo se necesitan unas pocas.

Proxy: permite proporcionar un sustituto o marcador de posición para otro objeto. Un proxy controla el acceso al objeto original, permitiéndote hacer algo antes o después de que la solicitud llegue al objeto original.

Patrones de Comportamiento: tratan con algoritmos y la asignación de responsabilidades entre objetos.

Command: convierte una solicitud en un objeto independiente que contiene toda la información sobre la solicitud. Esta transformación te permite parametrizar los métodos con diferentes solicitudes, retrasar o poner en cola la ejecución de una solicitud y soportar operaciones que no se pueden realizar.

Iterator: es un patrón de diseño de comportamiento que te permite recorrer elementos de una colección sin exponer su representación subyacente (lista, pila, árbol, etc.).

Observer: permite definir un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.

Strategy: permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.